

Πατάτα

- **Όριο χρόνου:** 10 λεπτά
- **Περιβάλλον:** μία GPU (≈16 GB VRAM), χωρίς διαδίκτυο
- **Μέγεθος λύσης:** `solution.ipynb` ≤ 1 MB
- **Αποθηκευτικός χώρος:** 5 GB

Πρόβλημα

Ο φίλος σας προτείνει να παίξετε ένα παιχνίδι μαντέματος. Εκείνος, ως κριτής, επιλέγει μία κρυφή λέξη από ένα σταθερό λεξιλόγιο και εσείς πρέπει να τη βρείτε σε το πολύ 30 γύρους. Σε κάθε γύρο, ο κριτής συγκρίνει δύο λέξεις και αναφέρει ποια είναι σημασιολογικά πιο κοντά στην κρυφή λέξη. Κάθε παιχνίδι ξεκινά από το σταθερό ζεύγος `lamp` vs `potato`, επειδή πρόκειται για δύο από τα αγαπημένα πράγματα του φίλου σας. Στη συνέχεια, το πρόγραμμά σας προτείνει μία νέα λέξη. Η λέξη που κερδίζει τη σύγκριση διατηρείται και συγκρίνεται με την επόμενη πρότασή σας. Κερδίζετε ένα παιχνίδι τη στιγμή που προτείνετε ακριβώς την κρυφή λέξη. Η αντιστοίχιση δεν κάνει διάκριση μεταξύ πεζών και κεφαλαίων. Κάθε λέξη που προτείνετε πρέπει να βρίσκεται στο `dataset/vocabulary.json`.

Υπάρχει ένα πλήρες παράδειγμα στο `solution.ipynb`, με το πρωτόκολλο και τη φόρτωση δεδομένων. Μπορείτε να αλλάξετε την κλάση `PublicEmbeddingPlayer`. Το πρόγραμμά σας αρχικοποιείται μία φορά και παίζει κάθε παιχνίδι σε μία ενιαία εκτέλεση: το πρωτόκολλο δημιουργεί ένα νέο `PublicEmbeddingPlayer` στην αρχή κάθε παιχνιδιού.

Ο Κριτής

Το πρόγραμμά σας στέλνει ένα αντικείμενο JSON στον Κριτή και ο Κριτής απαντά με ένα αντικείμενο JSON.

Ένα αναλυτικό παράδειγμα, στο οποίο η κρυφή λέξη εμφανίζεται μόνο για να εξηγηθεί το πρωτόκολλο:

```
Hidden word: shovel          Fixed opening: lamp vs potato

<- {"turn": 1, "winner_word": "potato", "verdict": "second", "word1": "lamp", "word2": "potato"}
-> {"new_word": "rock"}
<- {"turn": 2, "winner_word": "rock", "verdict": "second", "word1": "potato", "word2": "rock"}
-> {"new_word": "hammer"}
<- {"turn": 3, "winner_word": "hammer", "verdict": "second", "word1": "rock", "word2": "hammer"}
-> {"new_word": "shovel"}
-> {"status": "win"}
<- matches: game won
```

Οι γύροι αριθμούνται από 1 έως 30.

Οι επιλογές του `verdict` είναι `first`, που σημαίνει ότι η `word1` είναι πιο κοντά, `second`, που σημαίνει ότι η `word2` είναι πιο κοντά, ή `same`, που σημαίνει ότι και οι δύο λέξεις είναι εξίσου κοντά στην κρυφή λέξη.

Το `winner_word` είναι η λέξη που διατηρείται για την επόμενη σύγκριση. Σε μια ετυμηγορία `same`, παραμένει η πρώτη λέξη.

Σύνολο δεδομένων

Κοινά σε κάθε split:

- `dataset/vocabulary.json` — 1602 μοναδικές λέξεις με πεζά γράμματα. Η κρυφή λέξη είναι πάντα μία από αυτές.
- `dataset/public_embeddings.npy` — `float32`, σχήματος (1602, 2560). Η γραμμή `i` αντιστοιχεί στη λέξη `i` του λεξιλογίου. Αυτά είναι δημόσια embeddings· ο κριτής χρησιμοποιεί μια διαφορετική, ιδιωτική αναπαράσταση.

Τα split είναι σύνολα κρυφών λέξεων:

Split	Λέξεις	Απαντήσεις	Χρήση
<code>test_public</code>	120	✓ <code>dataset/test_public.json</code>	εκτέλεση της λύσης σας και αυτοαξιολόγηση
<code>test_leaderboard_a</code>	120	κρυφές	ζωντανός πίνακας κατάταξης
<code>test_leaderboard_b</code>	120	κρυφές	τελική κατάταξη

Δεν υπάρχει `split train` — τίποτα δεν προσαρμόζεται βάσει επισημασμένων γραμμών.

Παρεχόμενα μοντέλα

Μαζί με το πρόβλημα παρέχονται δύο προεκπαιδευμένα μοντέλα `embeddings`, τα οποία μπορούν να χρησιμοποιηθούν:

```
models/Qwen/Qwen3-Embedding-0.6B
[Qwen Embedding model card] (https://yastatic.net/s3/contest/ioai/3/01_Qwen_Qwen3-Embedding-0.6B_MODEL_CARD.html)
models/BAAI/bge-m3
[bge-m3 model card] (https://yastatic.net/s3/contest/ioai/3/02_BAAI_bge-m3_MODEL_CARD.html)
```

Και τα δύο πρέπει να φορτωθούν από την τοπική διαδρομή τους· ένα Hugging Face hub id όπως το "BAAI/bge-m3" ενεργοποιεί λήψη και αποτυγχάνει, επειδή η αξιολόγηση γίνεται εκτός σύνδεσης. Κάθε κατάλογος περιέχει ένα εκτελέσιμο `example.py` που παρουσιάζει την κλήση εκτός σύνδεσης.

Διαθέσιμες βιβλιοθήκες: `numpy`, `torch`, `sentence-transformers`. Δεν υπάρχει πρόσβαση στο διαδίκτυο, δεν επιτρέπονται λήψεις ούτε άλλα πακέτα.

Έξοδος

Καμία. Πρόκειται για διαδραστικό πρόβλημα: η λύση σας δεν γράφει αρχείο απαντήσεων· επικοινωνεί με τον κριτή μέσω `stdin/stdout`, όπως περιγράφεται παραπάνω.

Μετρική

Ένα παιχνίδι που επιλύεται στον γύρο t λαμβάνει βαθμολογία $1.0 - 0.02 \times \max(0, t - 10)$ · ένα παιχνίδι που δεν επιλύεται εντός 30 γύρων λαμβάνει βαθμολογία 0. Επομένως, οι γύροι 1-10 λαμβάνουν βαθμολογία 1.00, ο γύρος 20 λαμβάνει βαθμολογία 0.80 και ο γύρος 30 λαμβάνει βαθμολογία 0.60.

Η βαθμολογία σας στο πρόβλημα είναι η μέση βαθμολογία παιχνιδιού $\times 100$, μεταξύ 0.00 και 100.00.

Το όριο των 10 λεπτών αποτελεί έναν ενιαίο διαθέσιμο χρόνο που καλύπτει την εκκίνηση, την προετοιμασία και όλα τα 120 παιχνίδια του συνόλου ελέγχου.

Τρόπος υποβολής

1. Ανοίξετε το `solution.ipynb`, επεξεργαστείτε το `PublicEmbeddingPlayer` και εκτελέστε όλα τα κελιά για να βεβαιωθείτε ότι λειτουργεί.
2. Προαιρετικά, ελέγξτε το τοπικά: `python local_test.py solution.ipynb --limit 5`. Ο τοπικός κριτής χρησιμοποιεί τα *δημόσια* `embeddings`, επομένως η βαθμολογία του είναι μόνο ενδεικτική.
3. Αποθηκεύστε το `solution.ipynb`.
4. Ανοίξτε την καρτέλα Git στην αριστερή πλευρική γραμμή του JupyterLab.
5. Προσθέστε το `solution.ipynb` στο stage (το εικονίδιο + δίπλα του).
6. Εισαγάγετε ένα μήνυμα `commit` και κάντε κλικ στο Commit.
7. Κάντε κλικ στο σύννεφο με το βέλος προς τα επάνω για να κάνετε `push`.
8. Επιστρέψτε σε αυτή τη σελίδα Contest και κάντε κλικ στο Submit, με μήνυμα `commit` ίδιο με αυτό που έχετε εισαγάγει.

Υποβάλετε ακριβώς ένα αρχείο, με όνομα `solution.ipynb`, το οποίο να καλύπτει όλες τις απαραίτητες προετοιμασίες και την εξαγωγή συμπερασμάτων.